

```
from google.colab import drive
```

```
drive.mount("/content/drive")
```

↗ Mounted at /content/drive

```
from mlxtend.plotting import plot_decision_regions
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline
```

```
diabetes_data = pd.read_csv('/content/drive/MyDrive/diabetes.csv')
```

```
diabetes_data.head()
```

↗

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	A
0	6	148	72	35	0	33.6		0.627
1	1	85	66	29	0	26.6		0.351
2	8	183	64	0	0	23.3		0.672
3	1	89	66	23	94	28.1		0.167
4	0	137	40	35	168	43.1		2.288

```
diabetes_data.info(verbose=True)
```

↗

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Pregnancies         768 non-null   int64
1   Glucose             768 non-null   int64
2   BloodPressure       768 non-null   int64
3   SkinThickness       768 non-null   int64
4   Insulin             768 non-null   int64
5   BMI                 768 non-null   float64
6   DiabetesPedigreeFunction 768 non-null   float64
7   Age                 768 non-null   int64
8   Outcome             768 non-null   int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
diabetes_data.describe()
```



	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigr
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	

```
diabetes_data.describe().T
```



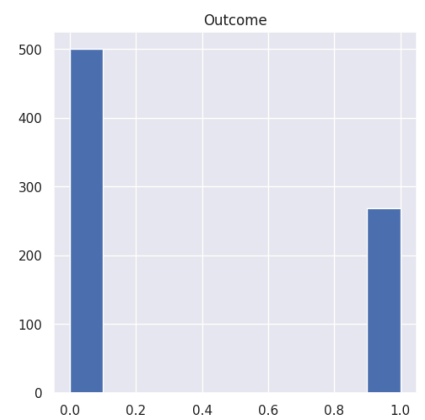
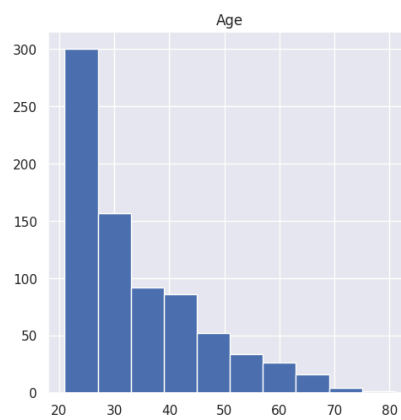
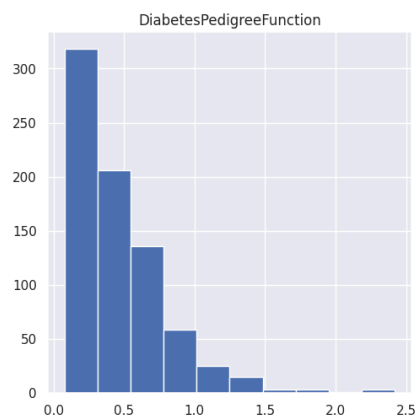
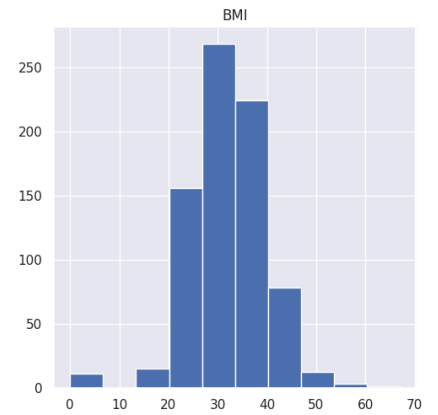
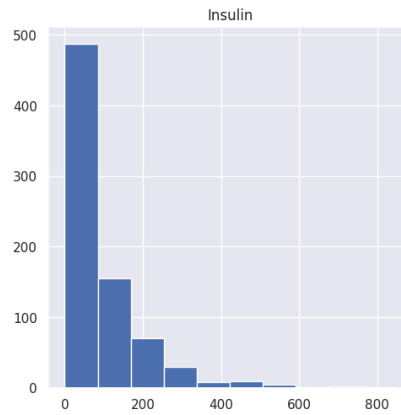
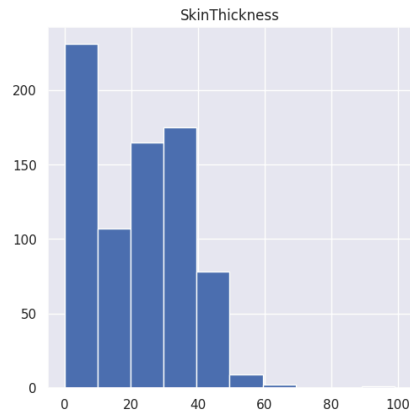
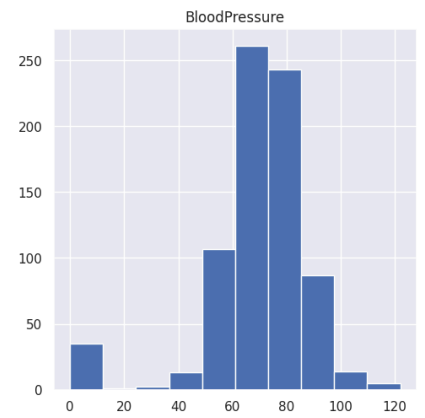
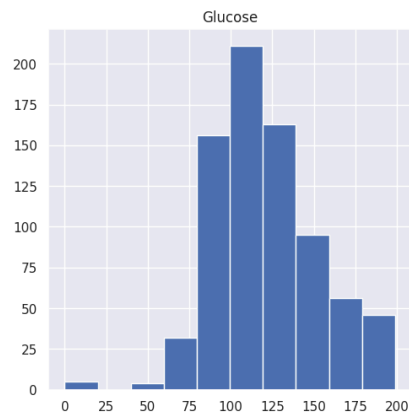
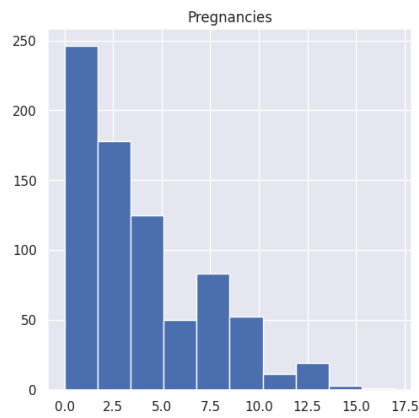
	count	mean	std	min	25%	50%	75%	max
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	3.0000	6.00000	17.00
Glucose	768.0	120.894531	31.972618	0.000	99.00000	117.0000	140.25000	199.00
BloodPressure	768.0	69.105469	19.355807	0.000	62.00000	72.0000	80.00000	122.00
SkinThickness	768.0	20.536458	15.952218	0.000	0.00000	23.0000	32.00000	99.00
Insulin	768.0	79.799479	115.244002	0.000	0.00000	30.5000	127.25000	846.00
BMI	768.0	31.992578	7.884160	0.000	27.30000	32.0000	36.60000	67.10
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.24375	0.3725	0.62625	2.42
Age	768.0	33.240885	11.760232	21.000	24.00000	29.0000	41.00000	81.00
Outcome	768.0	0.348958	0.476951	0.000	0.00000	0.0000	1.00000	1.00

```
diabetes_data_copy = diabetes_data.copy(deep = True)
diabetes_data_copy[['Glucose','BloodPressure','SkinThickness','Insulin','BMI']] = diabetes_data_co
print(diabetes_data_copy.isnull().sum())
```



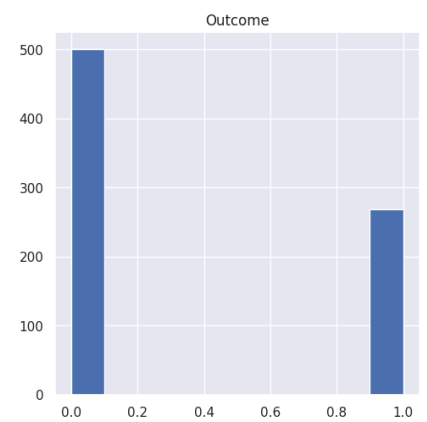
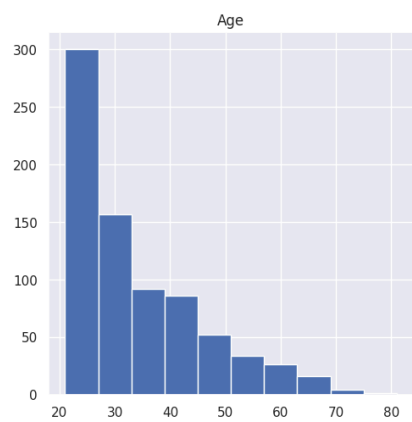
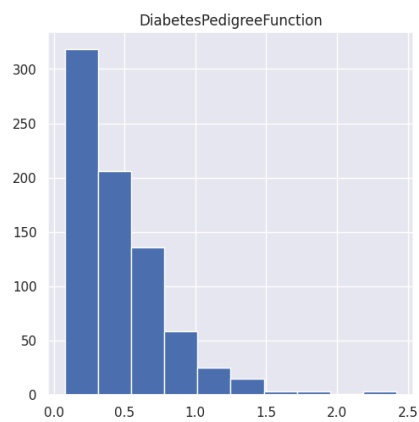
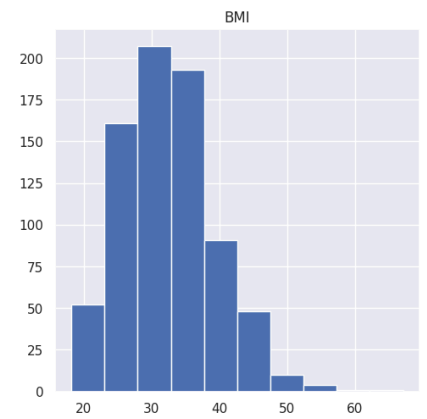
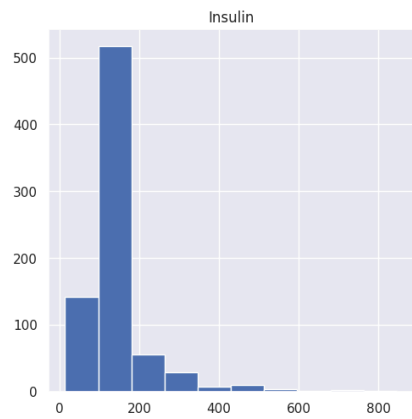
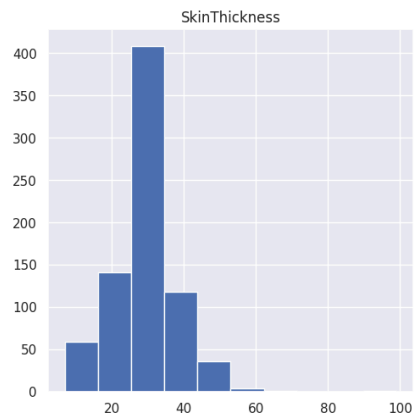
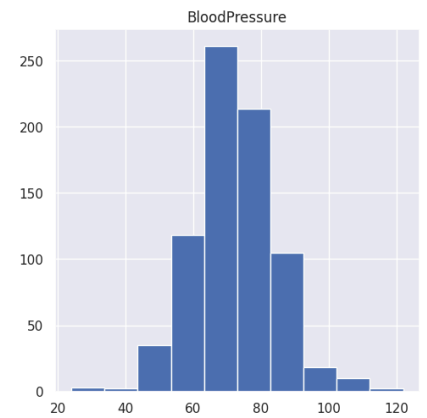
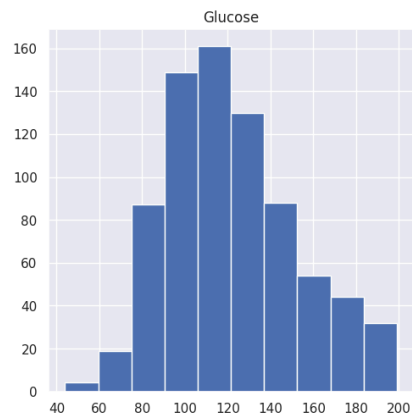
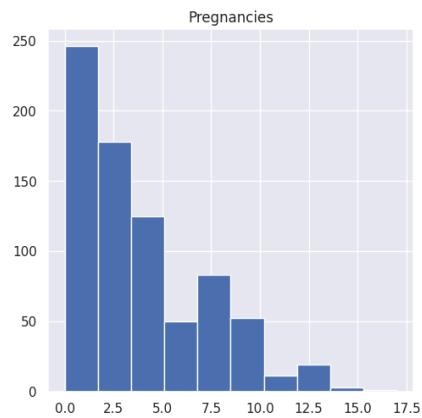
```
Pregnancies      0
Glucose           5
BloodPressure     35
SkinThickness    227
Insulin          374
BMI              11
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

```
p = diabetes_data.hist(figsize = (20,20))
```



```
diabetes_data_copy['Glucose'].fillna(diabetes_data_copy['Glucose'].mean(), inplace = True)
diabetes_data_copy['BloodPressure'].fillna(diabetes_data_copy['BloodPressure'].mean(), inplace = T
diabetes_data_copy['SkinThickness'].fillna(diabetes_data_copy['SkinThickness'].median(), inplace =
diabetes_data_copy['Insulin'].fillna(diabetes_data_copy['Insulin'].median(), inplace = True)
diabetes_data_copy['BMI'].fillna(diabetes_data_copy['BMI'].median(), inplace = True)
```

```
p = diabetes_data_copy.hist(figsize = (20,20))
```

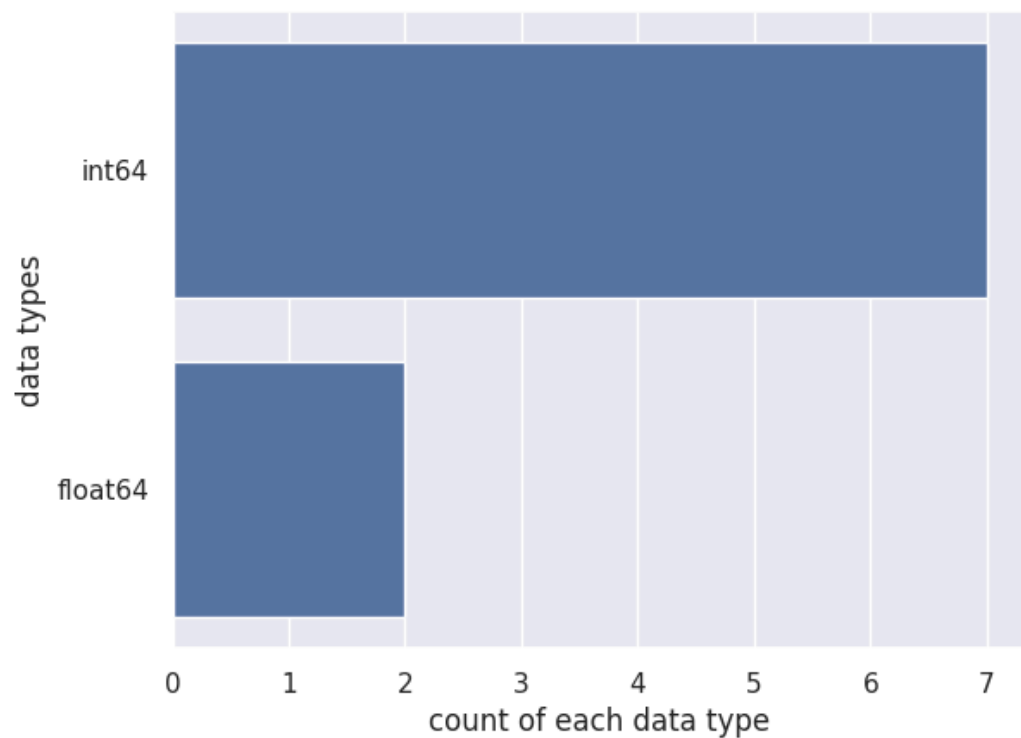


```
diabetes_data.shape
```

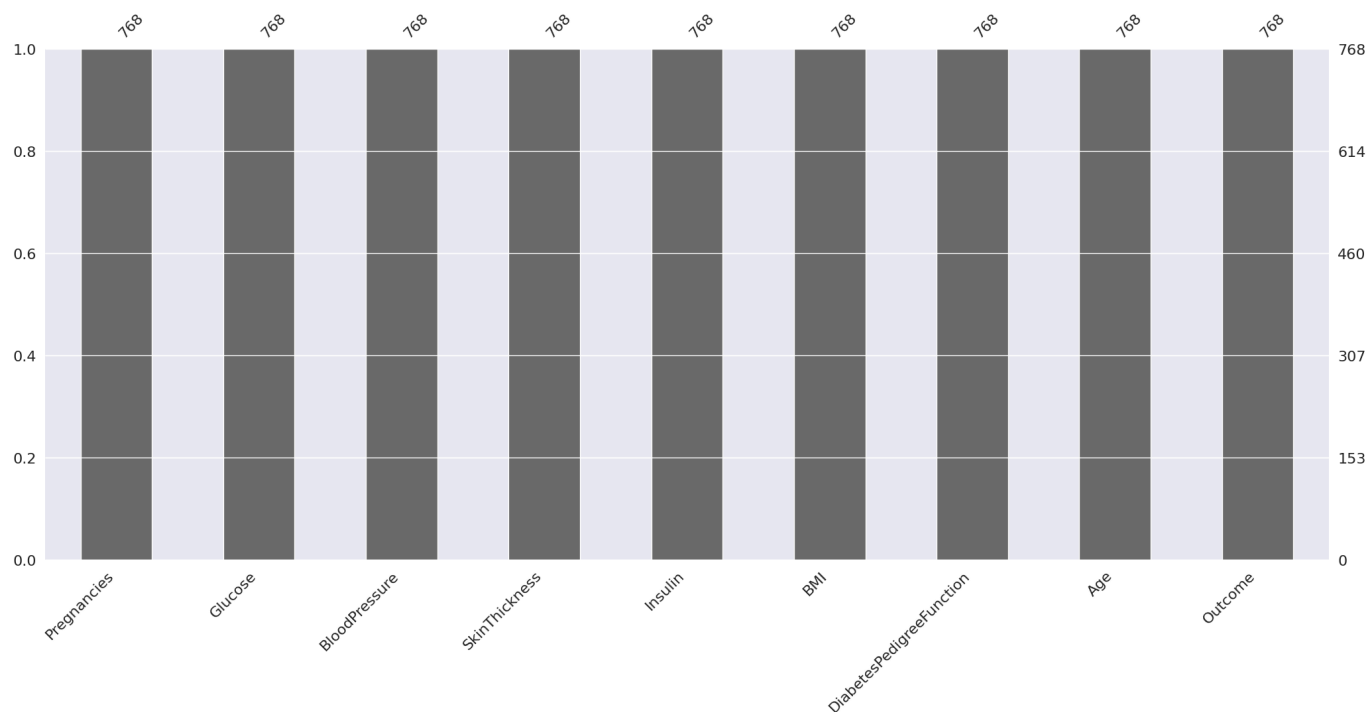


```
(768, 9)
```

```
sns.countplot(y=diabetes_data.dtypes ,data=diabetes_data)
plt.xlabel("count of each data type")
plt.ylabel("data types")
plt.show()
```



```
import missingno as msno
p=msno.bar(diabetes_data)
```

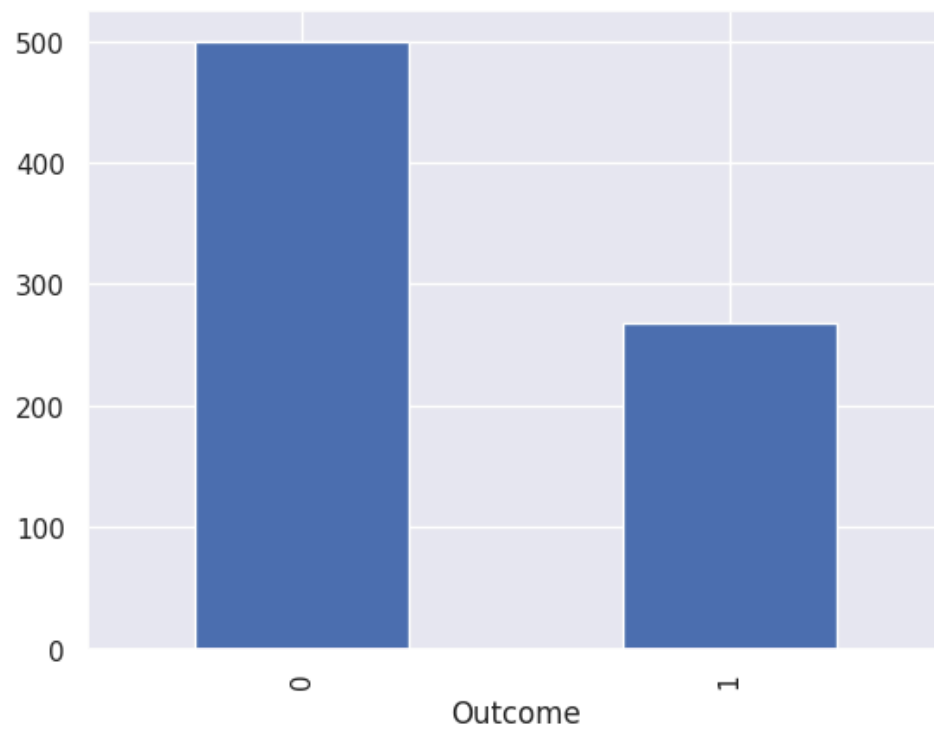


```
color_wheel = {1: "#0392cf",
                2: "#7bc043"}
colors = diabetes_data["Outcome"].map(lambda x: color_wheel.get(x + 1))
print(diabetes_data.Outcome.value_counts())
p=diabetes_data.Outcome.value_counts().plot(kind="bar")
```

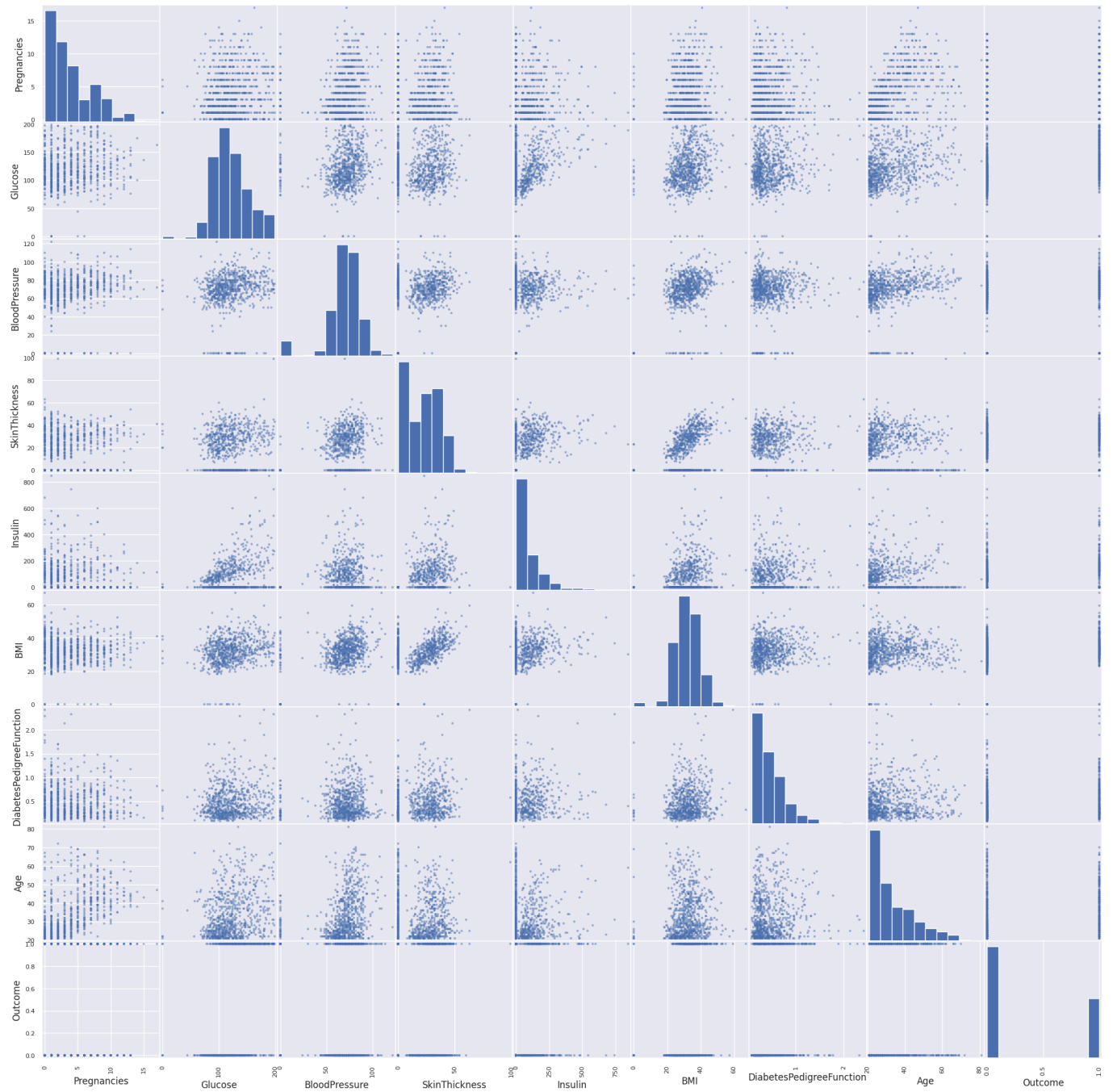


Outcome
0 500
1 268

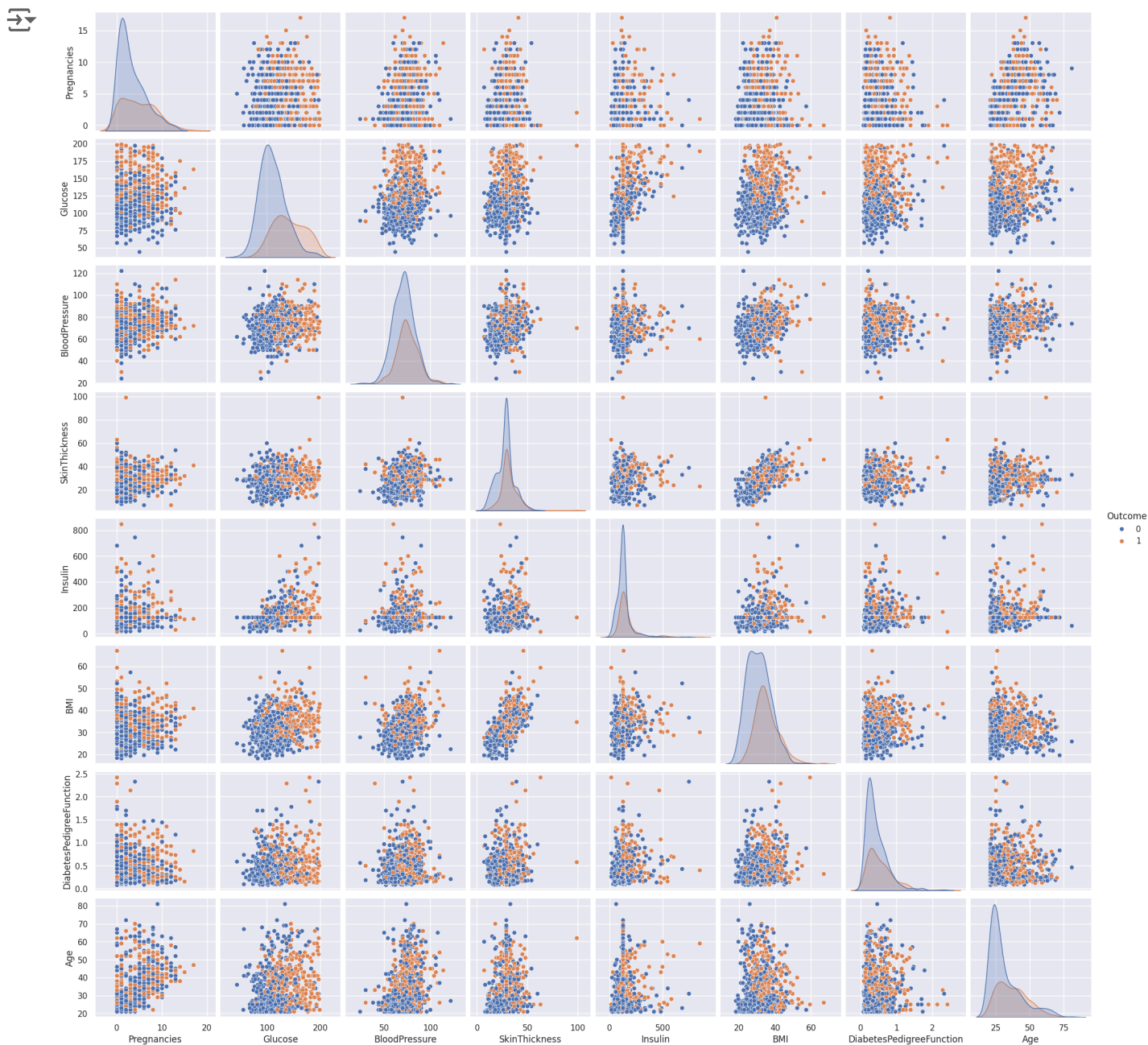
Name: count, dtype: int64



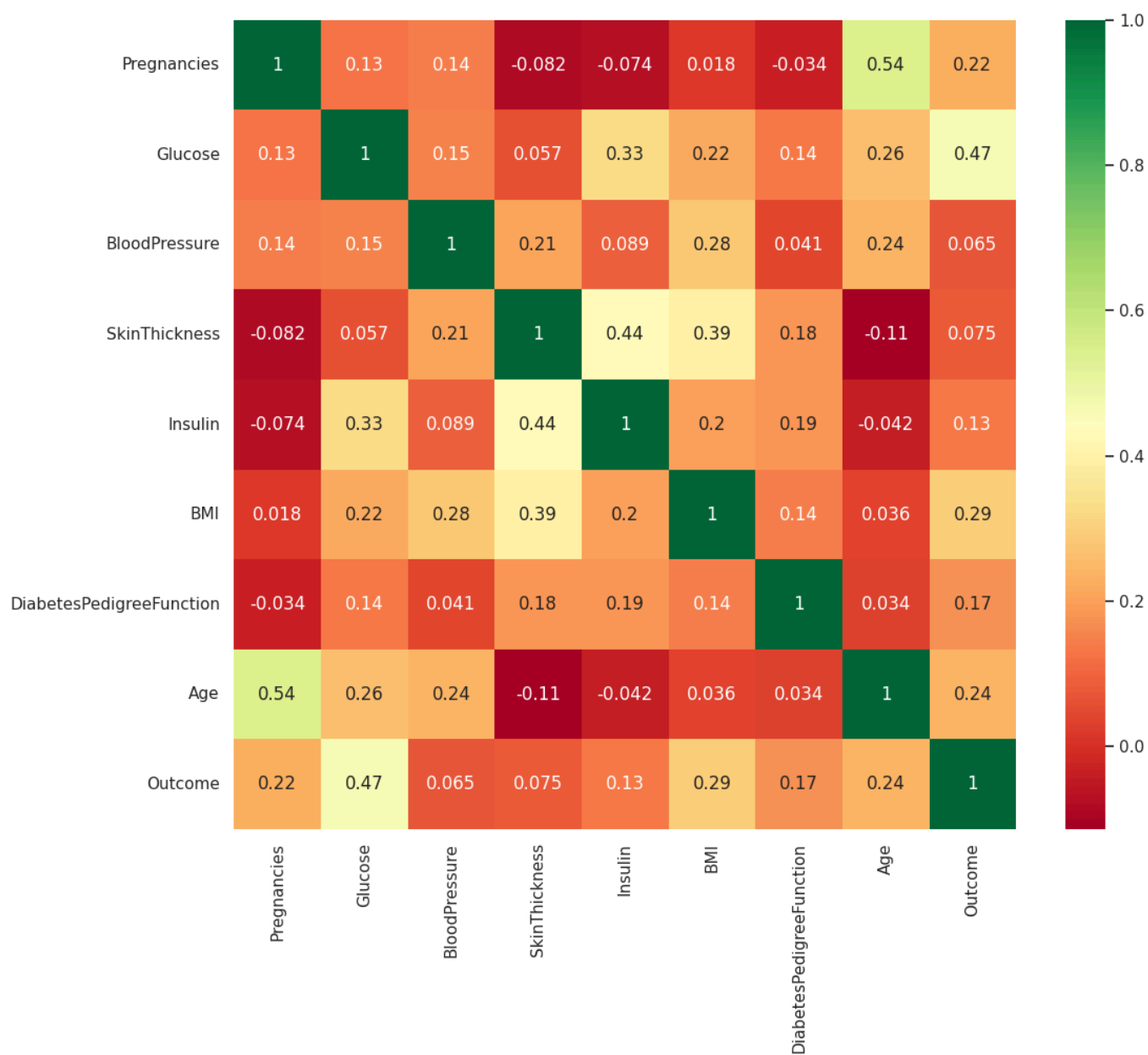
```
from pandas.plotting import scatter_matrix  
p=scatter_matrix(diabetes_data,figsize=(25, 25))
```



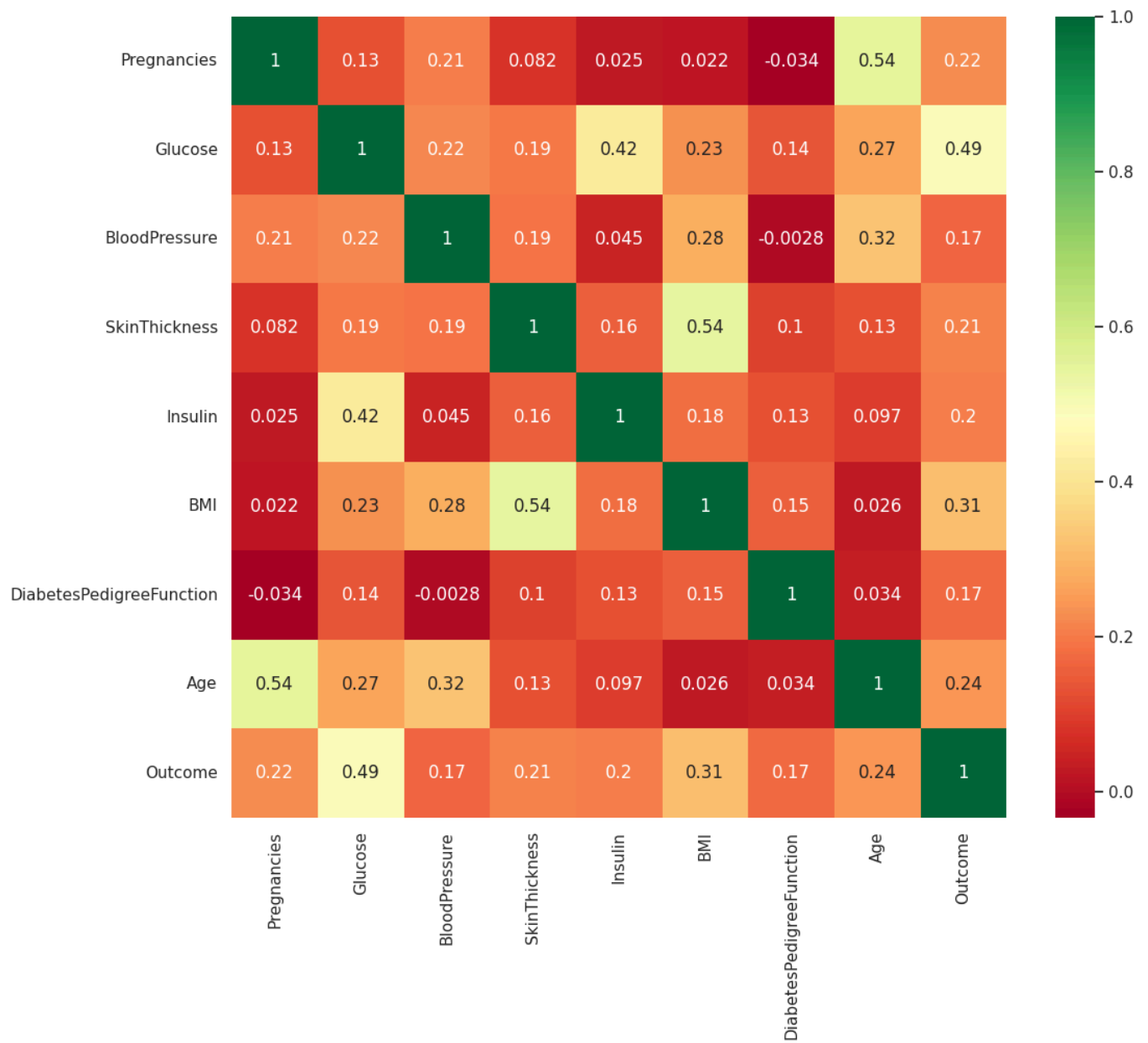
```
p=sns.pairplot(diabetes_data_copy, hue = 'Outcome')
```



```
plt.figure(figsize=(12,10))
p=sns.heatmap(diabetes_data.corr(), annot=True,cmap ='RdYlGn')
```



```
plt.figure(figsize=(12,10))
p=sns.heatmap(diabetes_data_copy.corr(), annot=True,cmap ='RdYlGn')
```



```
from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
X = pd.DataFrame(sc_X.fit_transform(diabetes_data_copy.drop(["Outcome"],axis = 1)),
                  columns=['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
                           'BMI', 'DiabetesPedigreeFunction', 'Age'])
```

```
X.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0	0.639947	0.865108	-0.033518	0.670643	-0.181541	0.166619	0.468
1	-0.844885	-1.206162	-0.529859	-0.012301	-0.181541	-0.852200	-0.365
2	1.233880	2.015813	-0.695306	-0.012301	-0.181541	-1.332500	0.604
3	-0.844885	-1.074652	-0.529859	-0.695245	-0.540642	-0.633881	-0.920
4	-1.141852	0.503458	-2.680669	0.670643	0.316566	1.549303	5.484

```
y = diabetes_data_copy.Outcome
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=1/3,random_state=42, stratify=y)
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
test_scores = []
train_scores = []
```

```
for i in range(1,15):
```

```
    knn = KNeighborsClassifier(i)
    knn.fit(X_train,y_train)
```

```
    train_scores.append(knn.score(X_train,y_train))
    test_scores.append(knn.score(X_test,y_test))
```

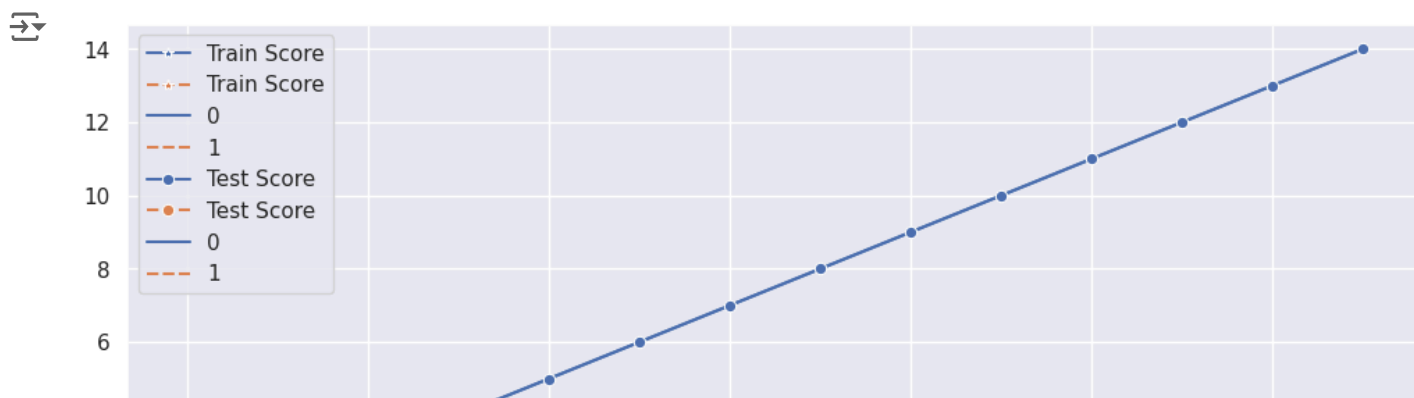
```
max_train_score = max(train_scores)
train_scores_ind = [i for i, v in enumerate(train_scores) if v == max_train_score]
print('Max train score {} % and k = {}'.format(max_train_score*100,list(map(lambda x: x+1, train_s
```

```
➡ Max train score 100.0 % and k = [1]
```

```
max_test_score = max(test_scores)
test_scores_ind = [i for i, v in enumerate(test_scores) if v == max_test_score]
print('Max test score {} % and k = {}'.format(max_test_score*100,list(map(lambda x: x+1, test_scor
```

```
➡ Max test score 76.5625 % and k = [11]
```

```
plt.figure(figsize=(12,5))
p = sns.lineplot(data=(range(1,15),train_scores),marker='*',label='Train Score')
p = sns.lineplot(data=(range(1,15),test_scores),marker='o',label='Test Score')
```



```
knn = KNeighborsClassifier(11)
```

```
knn.fit(X_train,y_train)
```

```
knn.score(X_test,y_test)
```

0.765625

```
value = 20000
```

```
width = 20000
```

```
plot_decision_regions(X.values, y.values, clf=knn, legend=2,
                      filler_feature_values={2: value, 3: value, 4: value, 5: value, 6: value, 7: value},
                      filler_feature_ranges={2: width, 3: width, 4: width, 5: width, 6: width, 7: width},
                      X_highlight=X_test.values)
```

```
plt.title('KNN with Diabetes Data')
```

```
plt.show()
```

